Practical SAT Solving

Lecture 8

Prof. Dr. Carsten Sinz

Hochschule Karlsruhe, University of Applied Sciences
Fakultät für Informatik und Wirtschaftsinformatik
Lehrgebiet Software-Entwicklung



CDCL Algorithm

Input: CNF Formula F
Output: `SAT` or `UNSAT`

Data: Assignment A $\leftarrow \emptyset$
Data: Clauses C $\leftarrow F$

```
1  while A is not complete do
2      UNIT PROPAGATION
3      if A falsifies a clause in C then
4          if branching is at level 0 then
5              return UNSAT
6          else
7              (conflict-clause, backtrack-level) ←
                CONFLICT-ANALYSIS
8              backtrack to backtrack-level
9              add conflict-clause to C
10             continue
11     if RESTART is triggered then
12         backtrack to level 0
13     if CLEANUP is triggered then
14         forget some of the learned clauses in C \ F
15     BRANCHING
16 return SAT
```

CDCL Algorithm

Input: CNF Formula F
Output: `SAT` or `UNSAT`

Data: Assignment A $\leftarrow \emptyset$
Data: Clauses C $\leftarrow F$

```
1  PREPROCESSING
2  while A is not complete do
3      UNIT PROPAGATION
4      if A falsifies a clause in C then
5          if branching is at level 0 then
6              return UNSAT
7          else
8              (conflict-clause, backtrack-level) ←
                CONFLICT-ANALYSIS
9              backtrack to backtrack-level
10             add conflict-clause to C
11     if RESTART is triggered then
12         backtrack to level 0
13     if CLEANUP is triggered then
14         forget some of the learned clauses in C \ F
15     BRANCHING
16 return SAT
```

Variable Selection in CDCL

Previous Decision Heuristics

Static: often computed only at root node of search

Global: based on whole CNF

- *Maximum Occurence in Minimum Sized Clauses (MOMS)*
  $S(x) = (f^*(x) + f^*(\overline{x})) \times 2^k + (f^*(x) \times f^*(\overline{x}))$
  ($f^*(x)$ is the number of occurrences of x in the smallest clauses)
- *Jeroslow-Wang:* $J(x) = \sum_{c \in F[x]} 2^{-|c|}$
  ($F[x] := \{c \in F | x \in c\}$)

Dynamic: take the current partial assignment into account

- *Dynamic Largest Individual Sum (DLIS)*
  $\arg\max_x (\max(|F[x]|, |F[\overline{x}]|))$
  (Several Variants: (R)DLIS, (R)DLCS)

VSIDS Heuristic

Variable State-Independent Decaying Sum (VSIDS)

- First presented in SAT solver Chaff (2001) [1]
- VSIDS (or a variant of it) implemented in most current CDCL solvers
- General approach: Compute score for each variable, select variable with highest score
  - Initial variable score is number of literal occurrences
  - New conflict clause c: Score is incremented for all variables in c
  - Periodically, divide all scores by a constant

Reasoning behind VSIDS

Make heuristics more "focused"

- try to find small unsatisfiable subsets
- prefer variables that occurred in a recent conflict

[1] Chaff: Engineering an efficient SAT solver (Moskewicz et al., 2001)

VSIDS Example

Initial F:

$\{x_1, x_4\}$
$\{x_1, \overline{x_3}, \overline{x_8}\}$
$\{x_1, x_8, x_{12}\}$
$\{x_2, x_{11}\}$
$\{\overline{x_7}, \overline{x_3}, x_9\}$
$\{\overline{x_7}, x_8, \overline{x_9}\}$
$\{x_7, x_8, \overline{x_{10}}\}$

Scores:

$4 : x_8$
$3 : x_1, x_7$
$2 : x_3$
$1 : x_2, x_4, x_9, x_{10}, x_{11}, x_{12}$

VSIDS Example

Initial F:

$\{x_1, x_4\}$
$\{x_1, \overline{x_3}, \overline{x_8}\}$
$\{x_1, x_8, x_{12}\}$
$\{x_2, x_{11}\}$
$\{\overline{x_7}, \overline{x_3}, x_9\}$
$\{\overline{x_7}, x_8, \overline{x_9}\}$
$\{x_7, x_8, \overline{x_{10}}\}$

Scores:

$4 : x_8$
$3 : x_1, x_7$
$2 : x_3$
$1 : x_2, x_4, x_9, x_{10}, x_{11}, x_{12}$

F with new learned clause added:

$\{x_1, x_4\}$
$\{x_1, \overline{x_3}, \overline{x_8}\}$
$\{x_1, x_8, x_{12}\}$
$\{x_2, x_{11}\}$
$\{\overline{x_7}, \overline{x_3}, x_9\}$
$\{\overline{x_7}, x_8, \overline{x_9}\}$
$\{x_7, x_8, \overline{x_{10}}\}$
$\{x_7, x_{10}, \overline{x_{12}}\}$ (new learned clause)

Scores:

$4 : x_8, x_7$
$3 : x_1$
$2 : x_3, x_{10}, x_{12}$
$1 : x_2, x_4, x_9, x_{11}$

Implementation of VSIDS

Possible: Keep list of variables sorted by score

Often implemented as Priority Queue

Operations:

- `insert_with_priority` (Backtracking)
- `pull_highest_priority_element` (Branching)
- `increase_key` (Bumping)

Often implemented as Binary Heap:

- Insert: $\mathcal{O}(\log n)$
- Delete: $\mathcal{O}(\log n)$
- Peek: $\mathcal{O}(1)$
- *Increase Key*: $\mathcal{O}(\log n)$

Implementation of VSIDS

Periodically divide scores to give priority to recently learned clauses

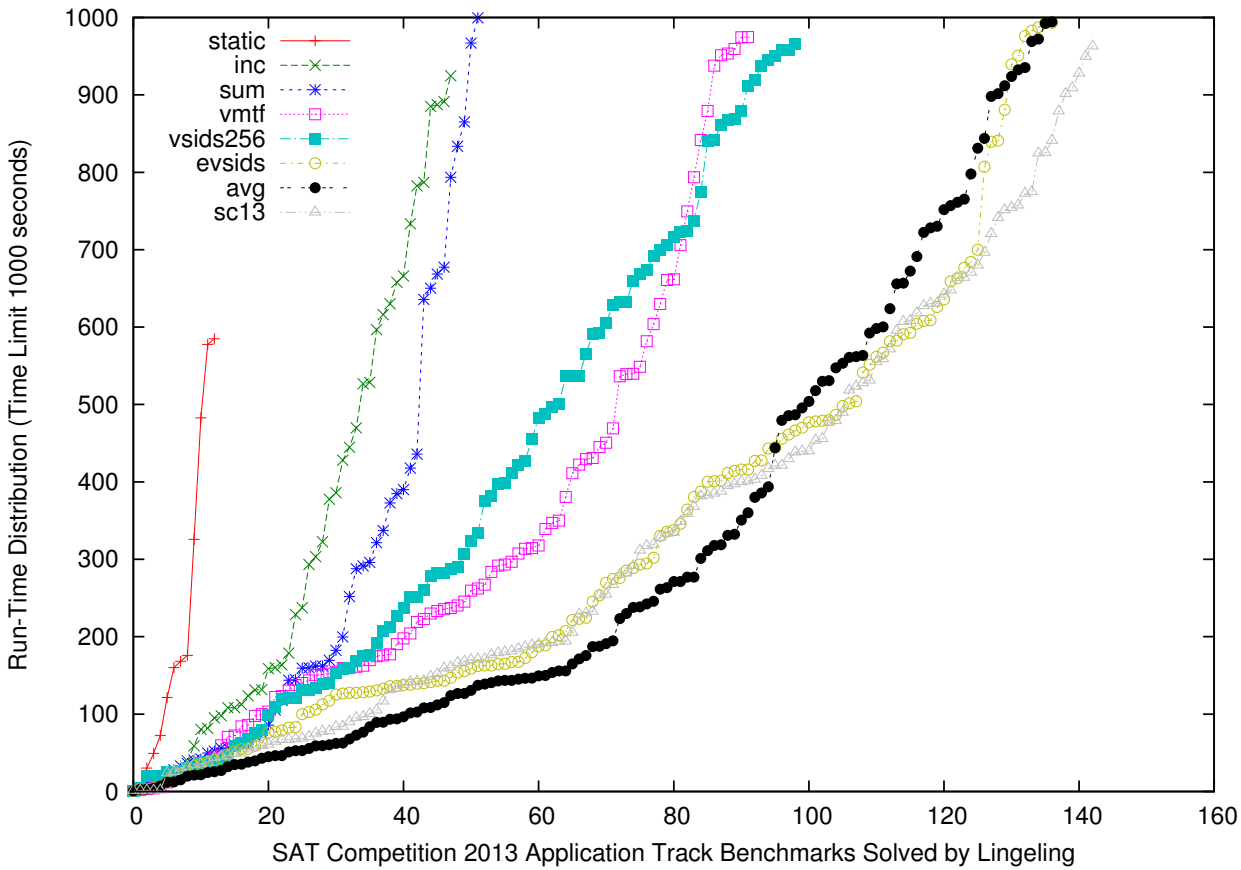
Variants

- Chaff (2001)
  - half scores every 256 conflicts ("decay")
  - sort priority queue after each decay only
- Berkmin (2002)
  - bump all literals in implication graph
  - divide scores by 4
- Minisat (2003)
  - exponential VSIDS (eVSIDS)

Alternatives

- Siege (2004): Variable Move To Front (VMTF)
- HaifaSAT (2008): Clause Move To Front (CMTF)

Comparison of Heuristics



Recent Trends and Further Material

Hybrid Approaches

Use other heuristic for a fixed *initial* period, then switch to VSIDS

- MapleCOMSPS (2016): Learning Rate-based Branching (LRB)
- Maple_LCM_Dist (2017): Distance Heuristic

Further Material

Survey on variants of VSIDS (+new heuristic ACIDS):
"Evaluating CDCL Variable Scoring Schemes" (Biere & Fröhlich, 2015)

Learned Clause Removal (Forgetting)

Problem: Too many learned clauses!

- Not all of them are helpful
- Unit Propagation gets slower
- Memory consumption increases

Solution: Clause Database Reduction a.k.a. Forgetting

- Size: Discard long clauses
- Least Recently Used (LRU): Discard clauses not involved in recent conflict clause generation (Clause Activity)
- Literal Block Distance (LBD): Discard clauses with high LBD.

Recent Trends: Hybrid LRU and LBD

Multi-Tier Clause Management

Manage clauses differently in three tiers:

1. `core`: permanently store clauses of LBD $\leq k$ (core-cut value)
2. `mid-tier`: clauses stay here if used in recent conflicts
3. `local`: manage fixed number of clauses by clause-activity

History

- `core` and `local` tier introduced in SWDiA5BY (Chanseok Oh, 2014)
- `mid-tier` introduced in CoMinisatPS (Chanseok Oh, 2015)
- "Between SAT and UNSAT: The Fundamental Difference in CDCL SAT" (Chanseok Oh, 2015)
- Note: MapleCOMSPS (2016) is a CoMinisatPS fork

Preprocessing

General Idea

- Try to reduce the size of the input formula by (polynomial time) simplification procedures.
- Assumption: Smaller problems are easier to solve

Note

Some solvers run simplification procedures periodically: *Inprocessing*

Subsumption

Subsumption

- Definition: A clause D subsumes a clause C iff $D \subseteq C$
- We also say that clause C is subsumed by D
- To check satisfiability, subsumed clauses are irrelevant: $\forall D \subseteq C, D \models C$

Example

$\{a, b\}$ subsumes $\{a, b, c\}$ and $\{a, b, d\}$

Self-Subsuming Resolution

Combination of Subsumption and Resolution

Self-Subsuming Resolution

- Definition: Let C, D be clauses and $\otimes_x$ the resolution operator on variable x. If $C \otimes D \subseteq C$ then C is said to be *self-subsumed by D with respect to x*.
- The resolvent of C and another clause D subsumes C

Example

- $C := \{\neg b, \neg e, f, \neg h\}$
- $D := \{\neg b, \neg e, \neg f\}$
- $E := C \otimes_f D = \{\neg b, \neg e, \neg h\}$
- The clause C is subsumed by the resolvent E

Variable Elimination

Variable Elimination (in given CNF F)

- Definition:
  - Let $S_x, S_{\overline{x}} \subset F$ be the sets of clauses containing x resp. $\overline{x}$
  - Let $R = \{C \otimes_x D \mid C \in S_x, D \in S_{\overline{x}}\}$ be the set of all resolvents on x
  - Modify F such that $F' := (F \setminus (S_x \cup S_{\overline{x}})) \cup R$
- Also called elimination by *clause distribution*
- The formulas F and F' are *satisfiability equivalent* (not equivalent)

Bounded Variable Elimination (BVE)

- Simulate "clause distribution" for all variables, but replace clauses only if number of clauses decreases (due to tautological resolvents)
- Most important simplification technique today

Variable Elimination and Gates

Property of Gate Encodings

- Resolving the clauses of a gate results in tautological clauses
- Example: Tseitin encoding of gate $x = AND(y, z)$ results in the clauses $\{\neg x, y\}, \{\neg x, z\}, \{x, \neg y, \neg z\}$

Idea

- Detect a gate, split formula F into $F = G \cup R$, where G are the gate-clauses and R the remaining clauses.
- Apply elimination by clause distribution:
  $S' = (G_x \cup R_x) \otimes (G_{\overline{x}} \cup R_{\overline{x}})$
- Clause distribution results in
  $S' = (G_x \otimes R_{\overline{x}}) \cup (R_x \otimes G_{\overline{x}}) \cup (R_x \otimes R_{\overline{x}})$
- Moreover, $(G_x \otimes R_{\overline{x}}) \cup (R_x \otimes G_{\overline{x}}) \models (R_x \otimes R_{\overline{x}})$. (Why?)
- We can replace S by the satisfiability-equivalent
  $(R_x \otimes G_{\overline{x}}) \cup (R_{\overline{x}} \otimes G_x)$

Blocked Clauses

Definition

A clause $(l \vee C)$ is blocked w.r.t. F by l if for every clause $(\neg l \vee D)$ in F the resolvent $(C \vee D)$ is a tautology.

Example

$F := (a \vee b) \wedge (a \vee \neg b \vee \neg c) \wedge (\neg a \vee c)$
First clause is not blocked, second is blocked by both a and $\neg c$, third is blocked by c.

Blocked Clause Elimination

Removal of an arbitrary blocked clause preserves satisfiability.
Blocked clause elimination (BCE) has a unique fixpoint.

Blocked Clauses and Gates

Left-Totality of Gate Encodings

Given a gate G with output variable o and its clausal encoding E, it holds that for every clause $C \in E$ either $o \in C$ or $\overline{o} \in C$ (part 1) and all resolvents in $E[o] \otimes_o E[\overline{o}]$ are tautologic (part 2).

Proof of Part 1

Assume that there is a clause $C \in E$ such that $o \notin \text{vars}(C)$. It follows that there exists an assignment to input variables which falsifies C for any assignment to o. This contradicts left-totality.

Proof of Part 2

Let R be a non-tautological resolvent in $E[o] \otimes_o E[\overline{o}]$. By Definition of resolution, it holds that $o \notin \text{vars}(R)$ and $E \models R$. This contradicts left-totality.

Failed Literal Elimination

Definition

If applying unit propagation on $F \wedge \{l\}$ derives UNSAT ($F \wedge \{l\} \vdash_{UP} \bot$), replace F by $F \wedge \{\neg l\}$.

Generalization

If $(F \setminus \{C\}) \wedge \neg C \vdash_{UP} \bot$, remove C from F.

Autarkies

Autarkies are a generalization of pure literals

Definition

Given a partial assignment σ and a formula F, a clause $C \in F$ is *touched by* σ if it contains the negation of a literal assigned in σ. A clause is *satisfied by* σ if it contains a literal assigned to *True* by σ. If all touched clauses are satisfied then σ is an *autarky*.

Autarky-based Clause Removal

- All clauses touched by an autarky can be removed
- Example: The partial assignment $\sigma = \{\neg a, \neg c\}$ is an autarky for $F := \{\neg a, b\}, \{\neg a, c\}, \{a, \neg b, \neg c\}, \{b, d\}, \ldots$ (more clauses without a and c)

Preprocessing Techniques that do not Reduce the Problem Size

- There are techniques, such as Bounded Variable Addition (BVA), that increase problem size.
- BVA is mainly based on the Extended Resolution rule.

Extended Resolution

Extended resolution adds a second rule to the resolution calculus, the Extension Rule. The idea is to introduce new variables as conjunction of existing literals, $x_{\text{new}} \leftrightarrow l_1 \wedge l_2$. As a rule for formulas in CNF:

$$\frac{}{(\neg x_{\text{new}} \vee l_1) \wedge (\neg x_{\text{new}} \vee l_2) \wedge (x_{\text{new}} \vee \neg l_1 \vee \neg l_2)}$$

- There are proofs with exponential size by Resolution, but polynomial size by Extended Resolution, e.g. pigeonhole formulas

Inprocessing

Idea: Interleave search and preprocessing

- Preprocessing can be extremely beneficial
- Most solvers in SAT competitions use bounded variable elimination, subsumption and self-subsuming resolution
- Problem: Many preprocessing techniques, though polynomial, require considerable time
- Possible Solutions:
  - "Preempt" (interrupt) preprocessing techniques after some time
  - Resume preprocessing between restarts
  - Limit preprocessing time in relation to search